

Informe Tecnico

Simulacion Logica de MIL-STD-1553 sobre SPI

Proyecto basado en Raspberry Pi Pico 2 W

8 de abril de 2026

Índice

1. Introduccion

El presente documento consolida y deja documentado el estado actual del desarrollo de una simulacion logica de MIL-STD-1553 sobre SPI utilizando dos placas Raspberry Pi Pico 2 W y un dashboard provisional en Flask. El objetivo general de esta etapa fue evolucionar desde un esquema de mini-tramas SPI simples hacia una representacion mucho mas fiel, a nivel logico, del comportamiento del estandar MIL-STD-1553, manteniendo todavia una implementacion de bajo riesgo, de facil depuracion y sin entrar aun en la capa fisica real del bus.

Durante esta fase se consolidaron tres piezas principales:

- un emisor **MIL-M** que construye palabras y mensajes logicos 1553;
- un receptor **MIL-E** que valida, interpreta contexto de mensaje y emite eventos estructurados por serial;
- un **dashboard Flask** provisional que consume la salida del esclavo y la presenta como sniffer de prueba mientras no se dispone de una Raspberry dedicada como host final.

La simulacion lograda no es aun un bus MIL-STD-1553 fisico real. No hay codificacion Manchester, ni sincronismos fisicos de palabra, ni front-end analogico 1553, ni tiempos de linea de 1 Mb/s medidos sobre el medio real. Sin embargo, a nivel conceptual y de flujo de protocolo, el sistema ya representa de manera util:

- palabras **COMMAND**, **DATA** y **STATUS**;
- mensajes **BC->RT**, **RT->BC**, **BROADCAST** y **MODE CODE**;
- chequeos de integridad;
- deteccion de anomalias de secuencia y de orden de mensaje;
- una salida estructurada apta para monitoreo y analisis en host.

2. Objetivos de la etapa

Los objetivos concretos perseguidos en esta etapa fueron los siguientes:

- reemplazar el esquema inicial de mini-tramas manuales por una representacion logica inspirada en MIL-STD-1553;
- mantener SPI como transporte interno entre placas, priorizando robustez y facilidad de prueba;
- separar responsabilidades entre emisor, receptor y host;
- aumentar el realismo del protocolo antes de migrar a la capa fisica real;
- dejar trazabilidad de errores, warnings y secuencias de mensaje;
- disponer de una herramienta de visualizacion temporal en PC mediante Flask.

3. Alcance y limites

3.1. Que se simula

Actualmente se simulan de manera logica los siguientes conceptos del estandar:

- estructura de palabra de comando;
- estructura de palabra de estado;
- palabra de datos de 16 bits;
- bit de direccion T/R;
- RT Address;
- Subaddress;
- Word Count;
- Mode Codes;
- direccion de broadcast;
- secuencias BC→RT y RT→BC;
- validacion de orden esperado en el receptor;
- estados simples del terminal remoto.

3.2. Que todavia no se simula fisicamente

Todavia no se implementaron los siguientes aspectos de una solucion MIL-STD-1553 real:

- codificacion Manchester II;
- sincronismos fisicos de palabra;
- temporizacion real de 20 us por palabra;
- ventana real de respuesta RT entre 4 us y 12 us;
- acople por transformador;
- front-end analogico diferencial;
- buses redundantes fisicos A/B;
- captura pasiva sobre una linea 1553 real.

4. Arquitectura resultante

La arquitectura quedo dividida en tres componentes claramente separados:

Bloque	Responsabilidad actual
MIL-M	Construccion y transmision de palabras y mensajes logicos MIL-STD-1553 sobre SPI.
MIL-E	Recepcion byte a byte, validacion de integridad, seguimiento de contexto de mensaje y emision de eventos estructurados por serial.
Flask-mil	Dashboard provisional en PC para consumir la salida serial del esclavo y actuar como sniffer temporal.

Esta separacion fue una decision deliberada de arquitectura. Se evito convertir al esclavo en un dashboard completo y tambien se evito sobrecargar al master con responsabilidades de analisis. El resultado es una base mas modular y escalable.

5. Formato de mini-trama SPI

Cada palabra logica MIL-STD-1553 se encapsula actualmente en una mini-trama SPI fija de 7 bytes:

[SYNC] [TYPE] [WORD_MSB] [WORD_LSB] [PARITY] [SEQ] [CHECKSUM]

5.1. Descripcion de campos

Campo	Tamano	Descripcion
SYNC	1 byte	Byte fijo de sincronizacion. Se utiliza 0xAA.
TYPE	1 byte	Tipo de palabra: 0x01 para COMMAND, 0x02 para DATA, 0x03 para STATUS.
WORD_MSB	1 byte	Byte mas significativo de la palabra logica de 16 bits.
WORD_LSB	1 byte	Byte menos significativo de la palabra logica de 16 bits.
PARITY	1 byte	Paridad impar calculada sobre los 16 bits de la palabra.
SEQ	1 byte	Contador secuencial por palabra transmitida.
CHECKSUM	1 byte	XOR de los 6 bytes anteriores como chequeo rapido de transporte.

5.2. Motivacion del formato

El formato se mantuvo deliberadamente pequeno y estable para:

- conservar la robustez del envio byte a byte;
- simplificar la recepcion en el esclavo;
- facilitar la depuracion por consola;
- permitir evolucionar la semantica del protocolo sin cambiar aun el transporte.

6. Modulo compartido de protocolo

Se creo y consolidó un modulo comun de protocolo:

- `mil1553_protocol.h`
- `mil1553_protocol.c`

Este modulo centraliza la logica compartida por master y slave, evitando duplicacion y asegurando consistencia en el tratamiento del protocolo.

6.1. Funciones incluidas

- calculo de checksum de transporte;
- calculo de paridad impar;
- construccion de `COMMAND WORD`;
- construccion de `DATA WORD`;
- construccion de `STATUS WORD`;
- encapsulado de mini-trama SPI;
- validacion de mini-trama recibida;
- decodificacion de `COMMAND WORD`;
- decodificacion de `STATUS WORD`;
- deteccion de `Mode Code`;
- deteccion de `Broadcast`;
- calculo de `Word Count` efectivo, incluyendo la regla 0 =>32.

6.2. Modelo de Command Word

La palabra de comando implementada sigue la estructura logica:

- `RT Address`: 5 bits;
- `T/R`: 1 bit;
- `Subaddress`: 5 bits;
- `Word Count` o `Mode Code`: 5 bits.

6.3. Modelo de Status Word

La palabra de estado implementada incluye los siguientes campos:

- RT Address;
- message error;
- instrumentation;
- service request;
- broadcast received;
- busy;
- subsystem flag;
- dynamic bus acceptance;
- terminal flag.

7. Evolucion funcional de MIL-M

7.1. Primera etapa del emisor

El master comenzo emitiendo secuencias simples del estilo:

COMMAND + DATA + STATUS

en forma repetitiva y con carga util incremental.

7.2. Estado actual del emisor

En su estado actual, **MIL-M** ya puede generar distintos tipos de mensaje logico:

- BC->RT;
- RT->BC;
- BROADCAST;
- MODE_CODE.

7.3. Mensajes BC a RT

La secuencia implementada es:

COMMAND + N DATA + STATUS

Esto simula un controlador de bus ordenando a un RT recibir una cantidad variable de palabras.

7.4. Mensajes RT a BC

La secuencia implementada es:

COMMAND + STATUS + N DATA

Esto representa a un RT respondiendo con estado y luego con carga util de datos.

7.5. Mensajes Broadcast

Se incorporo soporte para mensajes con RT Address = 31, tratados como broadcast. En esta simulacion se utiliza:

COMMAND + N DATA

sin palabra de estado posterior, respetando la logica general de evitar respuestas en broadcast.

7.6. Mode Codes

Tambien se incorporaron mensajes de Mode Code, usando Subaddress = 0 o 31. En la simulacion actual se emplea una variante simple del tipo:

COMMAND + STATUS

Esto permite comenzar a ejercitar el camino logico de control y supervision del RT.

7.7. Word Count variable

Se agrego soporte para word count variable por canal. El master puede transmitir mensajes con varias palabras de datos, en vez de limitarse a una sola.

7.8. Canales logicos de prueba

Se utilizaron canales de simulacion para hacer las pruebas mas legibles:

- temperatura;
- velocidad;
- presion;
- presion broadcast;
- mode sync.

7.9. Logging de MIL-M

El emisor imprime trazas de depuracion por cada palabra transmitida, indicando:

- secuencia SEQ;
- tipo de palabra;
- palabra cruda;
- paridad;
- frame SPI completo;
- interpretacion basica de la palabra.

Esto fue clave para validar visualmente la construccion correcta del protocolo.

8. Evolucion funcional de MIL-E

8.1. Primera etapa del receptor

El esclavo comenzo como receptor minimo de transporte. Su funcion era:

- reconstruir la mini-trama SPI;
- validar SYNC;
- validar paridad;
- validar CHECKSUM;
- imprimir la palabra en una salida serial legible.

8.2. Estado actual del receptor

En su estado actual, **MIL-E** hace bastante mas que validacion basica. Manteniendose dentro de su rol de receptor/transporte, ahora implementa:

- validacion de integridad por palabra;
- deteccion de saltos de secuencia SEQ;
- seguimiento de contexto de mensaje;
- deteccion de Mode Code;
- deteccion de Broadcast;
- conteo esperado y observado de palabras de datos;
- deteccion de anomalias de orden en la secuencia de palabras;
- emision de eventos estructurados para el host.

8.3. Eventos seriales emitidos

La interfaz serial del esclavo fue estandarizada en un conjunto de eventos aptos para ser consumidos por un host o dashboard:

Evento	Uso
RX_OK	Frame valido recibido y parseado.
RX_ERR	Frame invalido con detalle de errores de integridad.
RX_WARN	Warning de transporte o de protocolo.
RX_MSG_START	Inicio de un nuevo mensaje logico.
RX_DATA	Observacion de una palabra de datos dentro del mensaje activo.
RX_STATUS	Observacion de una palabra de estado dentro del mensaje activo.
RX_MSG_END	Fin del mensaje logico, con resultado OK o WARN.

8.4. Warnings logicos implementados

Ademas de la validacion de transporte, el esclavo puede detectar y reportar:

- SEQ_GAP;
- COMMAND_BEFORE_PREVIOUS_END;
- STATUS_IN_BROADCAST;
- STATUS_BEFORE_ALL_DATA;
- DUPLICATE_STATUS;
- DATA_AFTER_STATUS;
- DATA_OVERFLOW;
- ORPHAN_STATUS;
- ORPHAN_DATA;
- UNKNOWN_TYPE.

8.5. Flujo de arranque del esclavo

Se agregaron mensajes [INFO] al iniciar, y tambien se incorporo `fflush(stdout)` tras cada evento importante para que la salida llegue de forma mas estable al lector serial del host.

9. Dashboard Flask provisional

9.1. Motivacion

Como todavia no se dispone de una Raspberry dedicada para actuar como host final, se decidio adaptar la base existente en Flask que originalmente estaba orientada a ARINC, transformandola en un dashboard temporal para MIL.

9.2. Objetivo del dashboard

Su objetivo es actuar como sniffer provisional, permitiendo verificar rapidamente que:

- el esclavo recibe correctamente;
- los frames validos e invalidos se contabilizan;
- los mensajes logicos se reconstruyen;
- los warnings se muestran;
- la salida serial del esclavo es apta para analisis en host.

9.3. Cambios realizados en Flask

El backend `app.py` fue completamente adaptado para:

- leer por puerto serial;
- parsear eventos `RX...`;
- mantener buffers de consola, frames, warnings y mensajes;
- reconstruir mensajes a partir de `RX_MSG_START`, `RX_DATA`, `RX_STATUS` y `RX_MSG_END`;
- exponer estadisticas y endpoints JSON para la interfaz web.

9.4. Visualizaciones implementadas en la interfaz

La interfaz HTML fue rediseñada para mostrar:

- consola serial;
- frames validos e invalidos;
- warnings recientes;
- mensajes reconstruidos;
- resumen por tipo de palabra;
- resumen por direccion del mensaje;
- resumen por resultado;
- ultima palabra de estado observada.

9.5. Uso del filtro

El dashboard incluye un filtro textual simple que actua sobre consola y log. Por ejemplo, permite filtrar lineas que contengan:

- `RX_OK;`
- `RX_WARN;`
- `TYPE_NAME=COMMAND;`
- `MODE_CODE=1;`
- `RT=3.`

El sistema funciona tambien con el filtro vacio, mostrando toda la informacion disponible.

10. Pruebas realizadas y observaciones

10.1. Prueba integral de funcionamiento

Se realizo una prueba integral conectando:

- **MIL-M** como emisor;
- **MIL-E** como receptor;
- **Flask-mil** como visualizador sobre el puerto serial del esclavo.

El resultado observado fue positivo: el dashboard comenzo a mostrar trafico, indicando que el flujo master -¿SPI -¿slave -¿serial -¿Flask estaba funcional.

10.2. Warning observado: SEQ_GAP

Durante las pruebas se observo al menos un warning **SEQ_GAP**. Este warning significa que el receptor esperaba una secuencia consecutiva de mini-tramas, pero detecto un salto en el valor de **SEQ**.

En esta etapa del desarrollo, un warning aislado de este tipo al inicio de la captura no se considera critico. Las causas mas probables son:

- enganche del receptor o del Flask a mitad de una transmision ya iniciada;
- reinicio de una de las placas;
- desfasaje inicial entre master y slave.

La interpretacion actual es que un **SEQ_GAP** unico al principio no invalida la prueba si el resto del flujo permanece estable.

11. Validaciones implementadas

Las validaciones implementadas hasta ahora son tanto de transporte como de protocolo logico.

11.1. Validaciones de transporte

Validacion	Descripcion
SYNC	Verifica que el primer byte de la mini-trama sea 0xAA.
Paridad	Verifica la paridad impar sobre los 16 bits de palabra.
CHECKSUM	Verifica el XOR de los 6 primeros bytes del frame SPI.
SEQ	Verifica continuidad de secuencia entre palabras consecutivas.

11.2. Validaciones de mensaje

Chequeo	Descripcion
Orden de palabras	Verifica si un STATUS o un DATA aparecen en un orden inconsistente respecto del mensaje activo.
Cantidad de datos	Verifica si las palabras de datos vistas son menos o mas que las esperadas.
Broadcast	Verifica si aparece una palabra de estado donde logica y conceptualmente no deberia haber respuesta.
Mode Code	Permite distinguir mensajes especiales de control.
Estado huérfano	Detecta STATUS o DATA fuera de contexto de mensaje.

12. Estado tecnico alcanzado

En el estado actual del proyecto se puede afirmar que ya se dispone de una simulacion logica funcional y relativamente fiel de MIL-STD-1553 sobre SPI. Lo alcanzado incluye:

- mini-tramas estables de transporte;
- palabras COMMAND, DATA y STATUS;
- mensajes BC->RT, RT->BC, BROADCAST y MODE_CODE;
- Word Count variable;
- soporte de Word Count = 0 =>32 a nivel logico;
- deteccion de Mode Code;

- deteccion de Broadcast;
- validaciones de transporte;
- validaciones de contexto de mensaje;
- visualizacion provisional en Flask.

13. Limitaciones actuales

Persisten aun varias limitaciones, esperables para esta etapa:

- la capa fisica sigue siendo SPI y no un bus 1553 real;
- el receptor interpreta mensajes logicos simples, pero todavia no maneja RT->RT mediado por BC;
- no se implemento aun un paquete host mas rico por palabra;
- el dashboard actual es una herramienta provisional y no el host final;
- el contador SEQ utiliza un solo byte;
- no existe aun timestamp fino por palabra al estilo del informe tecnico 2.

14. Respuesta a una pregunta clave del proyecto

Una de las preguntas centrales durante esta etapa fue si todo este trabajo ya puede considerarse una simulacion de mini-tramas tipo STD-MIL con validacion. La respuesta actual es:

Si, a nivel logico y de transporte, el sistema ya puede considerarse una simulacion funcional de mini-tramas inspiradas en MIL-STD-1553 con su respectiva validacion.

Esto no significa aun compatibilidad fisica con un bus real, pero si representa una base valida y coherente para seguir evolucionando sin cambiar toda la arquitectura.

15. Proximos pasos recomendados

Los siguientes pasos tecnicamente razonables son:

1. incorporar soporte logico para mensajes RT->RT mediados por BC;
2. adaptar el receptor para interpretar correctamente esa secuencia;
3. enriquecer el dashboard Flask con filtros mas especificos por RT, SA y Mode Code;
4. definir el formato final de paquete host por palabra para la Raspberry dedicada;
5. migrar el host provisional a la Raspberry final cuando el hardware este disponible;
6. acercar gradualmente el sistema a la capa fisica real 1553.

16. Conclusion

El trabajo realizado hasta el momento permitio transformar un enlace SPI simple en una plataforma de simulacion logica de MIL-STD-1553 mucho mas ordenada, trazable y util para continuar el desarrollo. Se consolidaron componentes separados, un protocolo comun, mensajes mas realistas, validaciones de integridad y de contexto, y una visualizacion temporal en Flask que ya permite comprobar experimentalmente el funcionamiento de la cadena completa.

En terminos practicos, el proyecto se encuentra en un punto muy valioso: todavia es simple de depurar, pero ya representa suficiente semantica del estandar como para que el siguiente salto, hacia una implementacion fisica mas realista, pueda hacerse con mucho menor riesgo.